LeetCode HOT 100

PART 14

Greedy

贪心

77. 买卖股票的最佳时机 (No.121) EASY

给定一个数组 `prices` ，它的第 `i` 个元素 `prices[i]` 表示一支给定股票第 `i` 天的价格。

你只能选择 **某一天** 买入这只股票，并选择在 **未来的某一个不同的日子** 卖出该股票。设计一个算法来计算你所能获取的最大利润。

返回你可以从这笔交易中获取的最大利润。如果你不能获取任何利润，返回 `0` 。

示例 1：

> **输入：** [7,1,5,3,6,4]
> **输出：** 5
> **解释：** 在第 2 天（股票价格 = 1）的时候买入，在第 5 天（股票价格 = 6）的时候卖出，最大利润 = 6-1 = 5 。
> 注意利润不能是 7-1 = 6, 因为卖出价格需要大于买入价格；同时，你不能在买入前卖出股票。

示例 2：

> **输入：** prices = [7,6,4,3,1]
> **输出：** 0
> **解释：** 在这种情况下， 没有交易完成， 所以最大利润为 0。

提示：

- `1 <= prices.length <=` 10^5
- `0 <= prices[i] <=` 10^4

77. 买卖股票的最佳时机 (No.121) EASY

思路

从左往右记录股票历史最低点，然后不断计算收益更新最大值。

```python
class Solution:
    def maxProfit(self, prices: List[int]) -> int:
        low, res = prices[0], 0
        for price in prices:
            res = max(res, price - low)
            low = min(low, price)
        return res
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

78. 跳跃游戏 (No.55) MEDIUM

给你一个非负整数数组 `nums` ，你最初位于数组的 **第一个下标** 。数组中的每个元素代表你在该位置可以跳跃的最大长度。

判断你是否能够到达最后一个下标，如果可以，返回 `true` ；否则，返回 `false` 。

示例 1：

> **输入：** nums = [2,3,1,1,4]
> **输出：** true
> **解释：** 可以先跳 1 步，从下标 0 到达下标 1，然后再从下标 1 跳 3 步到达最后一个下标。

示例 2：

> **输入：** nums = [3,2,1,0,4]
> **输出：** false
> **解释：** 无论怎样，总会到达下标为 3 的位置。但该下标的最大跳跃长度是 0 ， 所以永远不可能到达最后一个下标。

提示：

- $1 <= nums.length <= 10^4$
- $0 <= nums[i] <= 10^5$

78. 跳跃游戏 (No.55) MEDIUM

思路

每步我只关心最远能跳到哪，因此需要维护一个最远点farthest。如果最远点更新速度赶不上遍历速度，说明已经来到一个之前跳不到的位置，就返回False。

```
class Solution:
    def canJump(self, nums: List[int]) -> bool:
        farthest = 0  # 能跳的最远点
        n = len(nums)
        for i in range(n):
            if i > farthest:  # 最远点赶不上遍历速度了就失败
                return False
            farthest = max(farthest, i + nums[i])
        return True
```

时间复杂度：$\mathcal{O}(n)$；空间复杂度：$\mathcal{O}(1)$

79. 跳跃游戏II (No.45) MEDIUM

给定一个长度为 `n` 的 **0 索引**整数数组 `nums`。初始位置在下标 0。

每个元素 `nums[i]` 表示从索引 `i` 向后跳转的最大长度。换句话说，如果你在索引 `i` 处，你可以跳转到任意 `(i + j)` 处：

- `0 <= j <= nums[i]` 且
- `i + j < n`

返回到达 `n - 1` 的最小跳跃次数。测试用例保证可以到达 `n - 1`。

示例 1:

> **输入：** `nums = [2,3,1,1,4]`
> **输出：** `2`
> **解释：** 跳到最后一个位置的最小跳跃数是 `2`。
> 从下标为 `0` 跳到下标为 `1` 的位置，跳 `1` 步，然后跳 `3` 步到达数组的最后一个位置。

示例 2:

> **输入：** `nums = [2,3,0,1,4]`
> **输出：** `2`

提示：

- $1 <= nums.length <= 10^4$
- `0 <= nums[i] <= 1000`
- 题目保证可以到达 `n - 1`

79. 跳跃游戏II (No.45) MEDIUM

思路

灵神对这道题的解释非常直观，这里借用他的说法，为了和Solution2的变量统一，这里改为farthest和next_far。遍历nums达到终点就像搭桥，每次我只关心我到底最远能到哪(贪心)，也就是下一座桥我最远能搭到哪。如果当前这座桥还没走完，我绝不更新res(因为求的是最小步数，除非走到桥的尽头，没必要半路多搭一座桥)。当站在桥的尽头时，此时不得不搭桥了，更新farthest和res。

如果题目还要求回溯路径，需要在每次更新next_far时记录下标，当i==farthest时的下标就是途径点。

直观来说，最优路径上的点，就是使得“下一步能跳最远”的点。这并不是下一步真要跳到最远，而是跳得远的点意味着选择的机会更多，从而更能找到最优的路径，因此更有价值。

```python
class Solution:
    def jump(self, nums: List[int]) -> int:
        res = 0
        farthest = 0  # 当前一座桥的最远点
        next_far = 0  # 下一座桥的最远点
        # 循环中，除非离开当前的桥，被迫再搭一座桥，否则就不更新res
        for i in range(len(nums)-1):
            next_far = max(next_far, i + nums[i])
            if i == farthest:  # 要离开当前的桥了，被迫搭桥
                farthest = next_far
                res += 1
        return res
```

时间复杂度：$O(n)$；**空间复杂度**：$O(1)$

80. 划分字母区间 (No.763) MEDIUM

给你一个字符串 `s` 。我们要把这个字符串划分为尽可能多的片段，同一字母最多出现在一个片段中。例如，字符串 `"ababcc"` 能够被分为 `["abab", "cc"]`，但类似 `["aba", "bcc"]` 或 `["ab", "ab", "cc"]` 的划分是非法的。

注意，划分结果需要满足：将所有划分结果按顺序连接，得到的字符串仍然是 `s` 。

返回一个表示每个字符串片段的长度的列表。

示例 1：

> **输入：** `s = "ababcbacadefegdehijhklij"`
> **输出：** `[9,7,8]`
> **解释：**
> 划分结果为 `"ababcbaca"`、`"defegde"`、`"hijhklij"` 。
> 每个字母最多出现在一个片段中。
> 像 `"ababcbacadefegde", "hijhklij"` 这样的划分是错误的，因为划分的片段数较少。

示例 2：

> **输入：** `s = "eccbbbbdec"`
> **输出：** `[10]`

提示：

- `1 <= s.length <= 500`
- `s` 仅由小写英文字母组成

80. 划分字母区间 (No.763) MEDIUM

思路

由于同一个字母不能被划分到不同区间，因此用alpha_map记录每个字母最后一次出现的位置。遍历s时，用right维护当前子串的右边界，右边界是当前子串所有字母里面最右边的那个。那么这题就和[79.跳跃游戏II]很像了，当我遍历到当前子串右边界时就切割，以划分出更多的区间，就像之前达到桥的最右端继续搭桥一样。

```
class Solution:
    def partitionLabels(self, s: str) -> List[int]:
        res, n = [], len(s)
        # alpha_map记录每个字母最后出现的位置
        alpha_map = {char: i for i, char in enumerate(s)}
        left, right = -1, -1  # left是上一个分割点，right是当前最右边界
        for i in range(n):
            # 当前子串最右边界更新
            right = max(right, alpha_map[s[i]])
            if right == i:
                res.append(right - left)
                left = right
        return res
```

时间复杂度：$\mathcal{O}(n)$；**空间复杂度**：$\mathcal{O}(|\Sigma|)$，$|\Sigma|$为s字符集大小。